



ACRON Methodology Part-7



در Part-6 ، فاز 5 تمام شد تعداد قدم ها 9 قدم

فاز 6: Cart Domain



فعال سازی و طراحی مدل های سفارش

کالبدشکافی معماری سفارشات (تفاوت Cart و Order چیست؟)

قبل از کدنویسی، باید یک مفهوم حیاتی در سیستم های مالی و فروشگاهی را درک کنیم: «تغییرپذیری در برابر تغییرناپذیری» (Mutable vs. Immutable).

- **سبد خرید (Cart):** یک موجودیت موقت و **تغییرپذیر** است. کاربر می تواند امروز یک لپ تاپ به سبدش اضافه کند، فردا قیمت لپ تاپ در سایت تغییر کند و وقتی کاربر به سبدش نگاه می کند، قیمت جدید را می بیند.
- **سفارش (Order):** یک **اسنپشات (عکس لحظه ای)** از تاریخ است و کاملاً **تغییرناپذیر** است. وقتی کاربر دکمه پرداخت را می زند و سفارش ثبت می شود، قیمت آن کالا در آن ثانیه، باید برای همیشه در دیتابیس ثبت (فریز) شود. اگر یک سال بعد قیمت لپ تاپ ۳ برابر شد، فاکتور کاربر (Order) نباید تغییر کند؛ او باید دقیقاً همان مبلغی را ببیند که در زمان خرید پرداخت کرده است.

به همین دلیل است که ما نمی توانیم فقط به جدول `Product` متصل بمانیم، بلکه باید فیلدی به نام `unit_price` (قیمت واحد) را در جدول `OrderItem` کپی و ذخیره کنیم.

1- فایل `config/settings/base.py` را باز کنید و اپلیکیشن `orders` را از کامنت خارج کنید:

```
] = INSTALLED_APPS
... #
, 'apps.carts'
, 'apps.orders' # این خط از کامنت خارج شود
[
```

2- فایل `apps/orders/models.py` را باز کنید و کدهای مهندسی شده ی زیر را بنویسید:

```

import uuid
from django.db import models
from apps.customers.models import Customer
from apps.products.models import Product

class Order(models.Model)
# ۱. تعریف وضعیت‌های مختلف یک سفارش با استفاده از TextChoices
class OrderStatus(models.TextChoices)
    'PENDING' = 'P', 'در انتظار پرداخت'
    'COMPLETED' = 'C', 'پرداخت موفق'
    'CANCELED' = 'X', 'لغو شده'

# ۲. شناسه یکتا و غیرقابل حدس برای پیگیری سفارش
id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)

# ۳. ارتباط با مشتری (سفارش برخلاف سبد خرید، حتماً صاحب دارد)
customer = models.ForeignKey(Customer, on_delete=models.PROTECT, related_name='orders')

# ۴. وضعیت فعلی سفارش
status = models.CharField(
    max_length=1,
    choices=OrderStatus.choices,
    default=OrderStatus.PENDING,
)

# ۵. زمان ثبت سفارش
created_at = models.DateTimeField(auto_now_add=True)

def __str__(self)
    "return f"Order {self.id} - {self.customer.user.username}"

class OrderItem(models.Model)
    order = models.ForeignKey(Order, on_delete=models.PROTECT, related_name='items')
    product = models.ForeignKey(Product, on_delete=models.PROTECT, related_name='order_items')
    quantity = models.PositiveSmallIntegerField()

# ۶. مهم‌ترین فیلد این فاز: فریز کردن قیمت!
unit_price = models.DecimalField(max_digits=10, decimal_places=2)

```

```
def __str__(self)
    return f"{self.product.name} (x{self.quantity})"
```

آموزش عمیق کدهای مدل سفارش (چرایی انتخاب‌ها)

1. چرا از `models.TextChoices` استفاده کردیم؟
در گذشته برنامه‌نویسان جنگو یک لیست ساده (تاپل) برای گزینه‌ها می‌نوشتند. اما کلاس `TextChoices` (که در نسخه‌های جدیدتر جنگو معرفی شد) بسیار شیء‌گراتر و تمیزتر است. این کار به ما اجازه می‌دهد در هر جای کد به جای اینکه بنویسیم `'if status == 'P'`، به صورت حرفه‌ای بنویسیم `if status == Order.OrderStatus.PENDING`. این کار جلوی خطاهای تایپی (Typo) را می‌گیرد و کد را به شدت خوانا می‌کند.

2. تفاوت `on_delete=models.PROTECT` با `CASCADE` در اینجا چیست؟
در سید خرید (`Cart`) ما از `CASCADE` استفاده کردیم، چون اگر کالا از فروشگاه پاک می‌شد، اشکالی نداشت از سید پرداخت‌نشده‌ی کاربر هم غیب شود.
اما در اینجا از `PROTECT` استفاده کردیم. فاکتور و سفارش، سند قانونی و مالی یک کسب‌وکار هستند. اگر ادمین تصمیم گرفت "گوشی آیفون ۱۳" را از دیتابیس پاک کند، دیتابیس باید ارور بدهد و بگوید: "شما حق ندارید این کالا را پاک کنید، چون در ۳ فاکتور گذشته فروخته شده است!" قانون طلایی سیستم‌های مالی: هیچ چیز مرتبط با تراکنش‌های گذشته نباید به سادگی قابل پاک شدن (`CASCADE`) باشد.

3. فیلد `unit_price` در `OrderItem`:
همان‌طور که در ابتدای صحبت‌ها توضیح دادم، این فیلد برای ثبت اسنپ‌شات قیمت است. بعداً در مرحله `Service Layer` (فاز ۸)، ما کدی می‌نویسیم که محتویات `CartItem` را می‌خواند و داخل `OrderItem` کپی می‌کند و در همان لحظه، قیمت فعلی کالا (`product.price`) را داخل `unit_price` ذخیره می‌کند.



انتقال تغییرات به دیتابیس

حالا که معماری قدرتمند فاکتورها را چیدیم، باید آن را روی MySQL اعمال کنیم.

2- در ترمینال بنویس:

```
python manage.py makemigrations orders
python manage.py migrate
```



ایستگاه منطقی بعدی ما، مفهوم `Service Layer` (لایه سرویس) است که در متدولوژی `acron` (فاز ۸) به آن اشاره شده.

لایه سرویس جایی است که ما منطق تبدیل "سبد خرید" به "سفارش" را می‌نویسیم. (تبدیل رکوردهای موقت به رکوردهای دائمی).

پنل مدیریت برای بخش سفارشات (فاکتورها) یکی از حساس‌ترین بخش‌های هر فروشگاه اینترنتی است. در اینجا، ما باید کاملاً با ذهنیت یک سیستم مالی کدنویسی کنیم: فاکتورها اسناد قانونی هستند و نباید به راحتی قابل دستکاری باشند.
بیا بیا ابتدا این امنیت را در پنل ادمین پیاده کنیم و سپس وارد مفهوم شگفت‌انگیز لایه سرویس (`Service Layer`) بشویم.



قدم اول: پیاده‌سازی پنل مدیریت سفارشات (ایمن و غیرقابل تغییر)

3- فایل `apps/orders/admin.py` را باز کنید و کدهای زیر را با دقت وارد کنید:

```
from django.contrib import admin
from .models import Order, OrderItem
```

```
# ۱. اینلاین برای آیتم‌های داخل فاکتور
class OrderItemInline(admin.TabularInline)
model = OrderItem
extra = 0 # ما نمی‌خواهیم ادمین دستی آیتم جدیدی به فاکتور اضافه کند

# قفل کردن فیلدها: ادمین نباید بتواند قیمت یا تعداد کالای فروخته شده را عوض کند!
readonly_fields = ['product', 'quantity', 'unit_price']

# جلوگیری از حذف کردن یک آیتم از وسط فاکتور ثبت شده
can_delete = False

# ۲. مدیریت اصلی فاکتورها
admin.register(Order)@
class OrderAdmin(admin.ModelAdmin)
list_display = ['id', 'customer', 'status', 'created_at']
list_filter = ['status', 'created_at']

# جستجو در روابط عمیق دیتابیس
search_fields = ['id', 'customer__user__username', 'customer__phone_number']

# قفل کردن فیلدهای اصلی فاکتور
readonly_fields = ['id', 'customer', 'created_at']

inlines = [OrderItemInline]

# جلوگیری از ساخت فاکتور دستی توسط ادمین (فاکتور فقط باید از طریق خرید کاربر ساخته شود)
def has_add_permission(self, request)
return False
```

کالبدشکافی خط به خط کدهای ادمین (چستی و چرایی)

- برخلاف گالری تصاویر (که ادمین باید می‌توانست عکس اضافه و کم کند)، در فاکتور مالی ادمین به `can_delete = False` و `extra = 0` را در `Add` و `Delete` هیچ وجه نباید بتواند آیتمی را به فاکتوری که پرداخت شده اضافه کند یا از آن حذف کند. این دو دستور، دکمه‌های فرانت‌اند پنل ادمین مخفی می‌کنند.
- `readonly_fields`: این یعنی ادمین در صفحه `quantity` و (قیمت فریز شده) `unit_price` ما فیلدهای حیاتی مثل `status` را فقط-خواندنی کردیم. این فیلدها خاکستری هستند و قابل ویرایش نیستند. تنها چیزی که ادمین حق دارد عوض جزئیات سفارش فقط می‌تواند اطلاعات را ببیند، اما فیلدها خاکستری هستند و قابل ویرایش نیستند. تنها چیزی که ادمین حق دارد عوض است. (مثلاً تغییر از "در انتظار" به "موفق") (وضعیت سفارش) `status` کند، فیلد `customer__user__username` برو `Order` است. علامت `__` (دو آندراین) به دیتابیس می‌گوید: "از جدول (ORM) این یکی از جادوهای جنگو: `customer__user__username` جستجو کن." این باعث می‌شود ادمین بتواند با تایپ کردن نام `username` و در فیلد `CustomUser` از آنجا برو به جدول `Customer` به جدول کاربری یک شخص، تمام فاکتورهای او را پیدا کند.
- `has_add_permission`: ما به طور کامل از پنل ادمین حذف کردیم! "Add Order" دکمه، `False` ما با اورراید کردن این متد و برگرداندن `has_add_permission`: چرایی: فاکتور باید نتیجه یک تراکنش سیستمی باشد، نه اینکه ادمین بنشیند و دستی فاکتور بسازد.

ورود به دنیای لایه سرویس (Service Layer)

اکنون که دیتابیس و پنل ادمین ما آماده است، می‌خواهیم API پرداخت و ثبت نهایی سفارش را بنویسیم. در پروژه‌های مبتدی، برنامه‌نویس تمام کدهای تبدیل سبد خرید به سفارش را مستقیماً داخل `views.py` می‌نویسد (الگوی ضدمعماری به نام Fat Views). اما در پروژه‌های بزرگ‌تر شبیه acron، ما از معماری سه‌لایه (Three-Tier Architecture) استفاده می‌کنیم.

چرا به لایه سرویس نیاز داریم؟

فرض کنید کاربر روی دکمه "تکمیل خرید" کلیک می‌کند. چه اتفاقاتی باید بیفتد؟

1. پیدا کردن سبد خرید کاربر.
 2. بررسی اینکه آیا کالاها هنوز در انبار موجودی دارند (`inventory > 0`)؟
 3. اگر موجودی نبود، خطا بدهد.
 4. اگر موجودی بود، یک رکورد جدید در جدول `Order` بسازد.
 5. روی تک‌تک `CartItem` ها حلقه بزند و آن‌ها را به `OrderItem` تبدیل کند.
 6. قیمت فعلی کالا را در `unit_price` فریز (کپی) کند.
 7. موجودی انبار (`inventory`) را کاهش دهد.
 8. سبد خرید (`Cart`) را برای همیشه پاک کند.
- اگر تمام این ۸ مرحله پیچیده را در `views.py` بنویسیم، کد ما غیرقابل خواندن، غیرقابل تست و به شدت کثیف می‌شود. لایه سرویس (Service Layer) دقیقاً برای همین به وجود آمده است: جدا کردن «منطق پردازشی کسب‌وکار» از «منطق دریافت درخواست HTTP».



قدم بعدی: ساخت اسکلت‌بندی لایه سرویس

4- در مسیر `/apps/orders` یک فایل جدید به نام `services.py` بسازید.

این فایل قرار است "مغز متفکر" بخش فروش ما باشد. ما یک کلاس و یک متد درون آن تعریف می‌کنیم که هیچ ارتباطی با `request`، `response`، یا `JSON` ندارد؛ بلکه فقط با دیتابیس و منطق خالص پایتون کار می‌کند.

```
from django.db import transaction
from rest_framework.exceptions import ValidationError
from apps.carts.models import Cart
from apps.orders.models import Order, OrderItem

class OrderService:

    @staticmethod
    @transaction.atomic
    def create_order_from_cart(cart_id, customer):
        """
        این سرویس یک سبد خرید را می‌گیرد و آن را به یک فاکتور قطعی تبدیل می‌کند.
        """
        # ۱. پیدا کردن سبد خرید به همراه آیتم‌ها و محصولاتش (برای جلوگیری از N+1)
        try:
            cart = Cart.objects.prefetch_related('items__product').get(
                id=cart_id)
```

```

except Cart.DoesNotExist
    raise ValidationError("سبد خرید یافت نشد یا قبلاً پرداخت شد  

    ه است.")

# ۲. اگر سبد خرید خالی بود، اجازه ساخت فاکتور نده!
if cart.items.count() == 0
    raise ValidationError("سبد خرید شما خالی است.")

# ۳. ساخت فاکتور اولیه (Header)
order = Order.objects.create(customer=customer)

# ۴. تبدیل تک تک آیتم‌های سبد به آیتم‌های فاکتور
[] = order_items_to_create
for cart_item in cart.items.all
    product = cart_item.product

# بررسی موجودی انبار در لحظه آخر
if product.inventory < cart_item.quantity
    raise ValidationError(f"موجودی محصول '{product.name}'  

    کافی نیست.")

# کسر از موجودی انبار
product.inventory -= cart_item.quantity
product.save

# آماده‌سازی آیتم فاکتور (دقت کنید قیمت همین الان فریز می‌شود)
order_items_to_create.append
    OrderItem
    ,order=order
    ,product=product
    ,quantity=cart_item.quantity
    # فریز کردن قیمت! unit_price=product.price
    (
    (

# ۵. ذخیره یکجای تمام آیتم‌ها در دیتابیس (بسیار بهینه‌تر از ذخیره  

    تک‌تک)
OrderItem.objects.bulk_create(order_items_to_create)

# ۶. حذف سبد خرید (چون تبدیل به فاکتور شد)
cart.delete

return order

```

کالبدشکافی لایه سرویس و دکوراتور `transaction.atomic@`

وقتی سرویسی می‌نویسیم که فقط مربوط به سید خرید است، جایش در `apps.carts.services` است. اما سرویس‌هایی در پروژه‌های واقعی وجود دارند که به هیچ Domain خاصی تعلق ندارند و همه اپلیکیشن‌ها به آن‌ها نیاز پیدا می‌کنند. اینجا دقیقاً نقطه استفاده از `core/services.py` است.

مثال‌های کاربردی برای `core/services.py` در آینده پروژه:

- سرویس ارسال پیامک (SMS Service): فردا می‌خواهید وقتی فاکتور ساخته شد، به کاربر پیامک برود. از طرفی وقتی در پروفایلش ثبت‌نام کرد هم پیامک برود. منطق اتصال به API کاوه‌نگار یا ملی‌پیامک در `core` نوشته می‌شود تا همه جا قابل فراخوانی باشد.
 - سرویس ایمیل (Email Notification): برای ارسال رسید فاکتور یا فراموشی رمز عبور.
 - تولید فایل PDF: متدی که یک دیکشنری پایتونی می‌گیرد و یک فایل PDF فاکتور یا گزارش خروجی می‌دهد.
 - ذخیره‌سازی ابری (Cloud Storage): اگر بخواهید در آینده تصاویر را به جای هارد سرور روی S3 آمازون یا آروان‌کلاد آپلود کنید.
- با قرار دادن این سرویس‌های عمومی در `core`، شما از قانون DRY (Don't Repeat Yourself) در بالاترین سطح معماری پیروی می‌کنید.



کجای کار هستیم؟

ما دیتابیس سفارشات (`models.py`) و مغز متفکر سیستم (`services.py`) را ساخته‌ایم. اما کلاینت (فرانت‌اند یا موبایل) چطور باید با این مغز متفکر صحبت کند؟ ما هنوز هیچ «دروازه»‌ای برای آن نساخته‌ایم. بنابراین، مرحله بعدی ما ساخت لایه API (View و Serializer) برای سفارشات است تا فرآیند تبدیل سید خرید به فاکتور را به اینترنت متصل کنیم.



قدم اول: ساخت Serializerهای سفارش

در اینجا ما به دو سریالایزر نیاز داریم: یکی برای نمایش فاکتور به کاربر (خروجی)، و دیگری برای دریافت درخواست ثبت سفارش (ورودی).

5- در مسیر `/apps/orders` یک فایل جدید به نام `serializers.py` بسازید

```
from rest_framework import serializers
from .models import Order, OrderItem
from apps.carts.models import Cart
from .services import OrderService

# ۱. سریالایزر نمایش آیتم‌های فاکتور
class OrderItemSerializer(serializers.ModelSerializer):
    # برای نمایش نام محصول به جای فقط آیدی آن
    product_name = serializers.CharField(source='product.name', read_only=True)

    class Meta:
        model = OrderItem
        fields = ['id', 'product_name', 'quantity', 'unit_price']

# ۲. سریالایزر نمایش کل فاکتور
class OrderSerializer(serializers.ModelSerializer):
    items = OrderItemSerializer(many=True, read_only=True)

    class Meta:
        model = Order
        fields = ['id', 'status', 'created_at', 'items']
```

```
# ۳. سریالایزر عملیاتی: فقط برای دریافت آیدی سبد خرید و ساخت فاکتور
class CreateOrderSerializer(serializers.Serializer)
    cart_id = serializers.UUIDField

    def validate_cart_id(self, cart_id)
        # بررسی اینکه آیا این سبد خرید اصلاً وجود دارد؟
        if not Cart.objects.filter(id=cart_id).exists
            raise serializers.ValidationError
                ("سبد خرید نامعتبر است یا قبلاً پرداخت شده است.")
        return cart_id

    def save(self, **kwargs)
        cart_id = self.validated_data['cart_id']

        # استخراج مشتری از ریکوئست (کاربر باید لاگین باشد)
        # ما request را از طریق context از سمت View به اینجا پاس میدهیم
        customer = self.context['request'].user.customer

        # فراخوانی لایه سرویس که در مرحله قبل ساختیم!
        order = OrderService.create_order_from_cart(cart_id=cart_id,
            customer=customer)

        return order
```



قدم دوم: ساخت View یا Controller

حالا باید یک ViewSet بسازیم که درخواست‌های HTTP را مدیریت کند. در اینجا برخلاف سبد خرید، کاربر حتماً باید لاگین کرده باشد تا بتواند سفارش ثبت کند.

-6 فایل `apps/orders/views.py` را باز کرده و کدهای زیر را وارد کنید:

```
from rest_framework.viewsets import GenericViewSet
from rest_framework.mixins import CreateModelMixin, RetrieveModelMixin, ListModelMixin
from rest_framework.permissions import IsAuthenticated
from drf_spectacular.utils import extend_schema_view, extend_schema
from .models import Order
from .serializers import OrderSerializer, CreateOrderSerializer

@extend_schema_view
    create=extend_schema(summary="تبدیل سبد خرید به سفارش (فاکتور)", tags=['Orders'])
    list=extend_schema(summary="لیست سفارشات کاربر", tags=['Order'])
```

```
,('s']
tags=['Order', "جزئیات یک سفارش"]=summary) retrieve=extend_schema
,('s']
(
class OrderViewSet(CreateModelMixin, RetrieveModelMixin, ListModelMix
:in, GenericViewSet)
"""
ویست مدیریت سفارشات مشتری.
دقت کنید که متدهای آپدیت و حذف مسدود شده اند، زیرا فاکتور قابل تغیی
ر نیست.
"""
# فقط کاربران لاگین شده حق دسترسی دارند
permission_classes = [IsAuthenticated]

# هر کاربر فقط باید فاکتورهای خودش را ببیند، نه دیگران را!
: def get_queryset(self)
user = self.request.user

# جلوگیری از خطای کاربرانی که هنوز پروفایل Customer ندارند
: if hasattr(user, 'customer')
return Order.objects.prefetch_related('items__product').f
ilter(customer=user.customer)
() return Order.objects.none

# انتخاب سریالایزر بر اساس نوع متد (دریافت یا ثبت)
: def get_serializer_class(self)
: if self.request.method == 'POST'
return CreateOrderSerializer
return OrderSerializer

# ارسال آبجکت request به سریالایزر برای دسترسی به اطلاعات کاربر
: def get_serializer_context(self)
return {'request': self.request}
```



قدم سوم: اتصال URLها

برای اینکه این API در دسترس قرار بگیرد، باید آن را به سیستم روتر جنگو متصل کنیم.

7- در مسیر `/apps/orders` فایل `urls.py` را ایجاد کنید:

```
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import OrderViewSet

() router = DefaultRouter
router.register('orders', OrderViewSet, basename='orders')
```

```
] = urlpatterns
,path('', include(router.urls))
[
```

8- این مسیر را در `apps/api/urls.py` (روتر مرکزی) ثبت کنید:

```
] = urlpatterns
... # مسیرهای قبلی ...
,path('', include('apps.carts.urls'))
,path('', include('apps.orders.urls'))
[
```

معماری این بخش چگونه کار می‌کند؟ (Flow)

۱. کاربر در فرانت‌اند دکمه "ثبت سفارش" را می‌زند.
 ۲. یک درخواست `POST` حاوی توکن احراز هویت و `cart_id` به سرور می‌آید.
 ۳. ویوی ما چک می‌کند کاربر لاگین است (`IsAuthenticated`).
 ۴. دیتا به `CreateOrderSerializer` می‌رود و ولیدیت می‌شود.
 ۵. سربالایزر به `OrderService` (لایه سرویس) دستور می‌دهد: "این سید خرید را بگیر و برای این مشتری فاکتور کن."
 ۶. لایه سرویس با دیتابیس درگیر می‌شود، سید را پاک می‌کند، کالاها را به فاکتور می‌برد و قیمت‌ها را فریز می‌کند.
 ۷. در نهایت یک فاکتور شیک ساخته شده و کد `Created 201` به کلاینت برمی‌گردد.
- با این کدها، چرخه اصلی فروشگاه (از دیدن محصول تا ثبت فاکتور) از نظر منطقی کامل شده است.



رفع باگ‌های منطقی در همان لایه‌ای که کشف می‌شوند (مثل همین قفل شدن موجودی انبار)، از انباشته شدن بدهی فنی (Technical Debt) در آینده جلوگیری می‌کند.

برای بازگرداندن موجودی انبار، در سیستم‌های مقیاس‌پذیر و به خصوص داشبوردهای پیشرفته مدیریت انبار، معماری استاندارد استفاده از تسک‌های پس‌زمینه با ابزارهایی مانند Celery و Redis است تا سیستم به صورت کاملاً خودکار و در بک‌گراند فاکتورها را بررسی کند. اما از آنجایی که در این فاز هنوز Celery را به پروژه متصل نکرده‌ایم، منطق «بررسی در لحظه» (Just-in-Time) را در لایه سرویس پیاده‌سازی می‌کنیم.



بخش اول: پیاده‌سازی منطق انقضای ۱۵ دقیقه‌ای فاکتور

ما باید دو متد جدید به «مغز متفکر» سفارشات اضافه کنیم: یکی برای لغو فاکتور و بازگرداندن موجودی انبار، و دیگری برای بررسی زمان سپری شده.

9- فایل `apps/orders/services.py` را باز کرده و این تغییرات را به کلاس `OrderService` اضافه کنید:
ابتدا این ایمپورت‌ها را به بالای فایل اضافه کنید:

```
from django.utils import timezone
from datetime import timedelta
```

سپس کدهای زیر را به داخل کلاس `OrderService` (زیر متد قبلی) اضافه کنید:

```
staticmethod@
transaction.atomic@
: def cancel_expired_order(order)
"""
```

```

این متد فاکتور را لغو کرده و موجودی کالاها را به انبار برمیگرد
اند.
"""
# اگر وضعیت فاکتور چیزی غیر از "در انتظار پرداخت" است، کاری ن
کن
:if order.status != Order.OrderStatus.PENDING
return False

# حلقه روی تمام آیتمهای فاکتور برای بازگرداندن موجودی
# استفاده از select_related برای جلوگیری از مشکل N+1 در ارتبا
ط با جدول Product
:for item in order.items.select_related('product')
product = item.product
product.inventory += item.quantity
()product.save

# تغییر وضعیت فاکتور به لغو شده
order.status = Order.OrderStatus.CANCELED
()order.save
return True

staticmethod@
:def validate_order_for_payment(order_id)
"""
این متد قبل از ارسال کاربر به درگاه بانکی فراخوانی میشود
تا بررسی کند آیا هنوز برای پرداخت فرصت دارد یا خیر.
"""
:try
order = Order.objects.get(id=order_id)
:except Order.DoesNotExist
raise ValidationError("سفارش یافت نشد.")

:if order.status == Order.OrderStatus.COMPLETED
raise ValidationError("این سفارش قبلاً پرداخت شده است.")

:if order.status == Order.OrderStatus.CANCELED
raise ValidationError("این سفارش لغو شده است.")

# محاسبه زمان انقضا (زمان ثبت فاکتور + ۱۵ دقیقه)
expiration_time = order.created_at + timedelta(minutes=15)

# مقایسه با زمان حال
:if timezone.now() > expiration_time
# فراخوانی متد بازگرداندن موجودی به انبار
OrderService.cancel_expired_order(order)
raise ValidationError("زمان ۱۵ دقیقه‌ای پرداخت به پایان رس

```

یده و سفارش به دلیل اتمام مهلت لغو شد."

```
return order
```

با این معماری، هر زمان که بخواهیم کاربر را به درگاه بانکی متصل کنیم، ابتدا متد `validate_order_for_payment` را صدا می‌زنیم. اگر زمان گذشته باشد، سیستم خودکار موجودی را به انبار برمی‌گرداند و جلوی پرداخت را می‌گیرد.



بخش دوم: تکمیل پروفایل کاربری (Address و Customer Profile)

10- به‌روزرسانی مدل‌ها: فایل `apps/customers/models.py` را باز کنید: ما مدل `Customer` را گسترش می‌دهیم و مدل جدیدی به نام `Address` می‌سازیم (چون یک مشتری می‌تواند چندین آدرس داشته باشد: خانه، محل کار و...).

```
from django.db import models
from django.conf import settings

class Customer(models.Model)
    user = models.OneToOneField(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
    phone_number = models.CharField(max_length=15, unique=True, null=True, blank=True)
    birth_date = models.DateField(null=True, blank=True)

    def __str__(self)
        "return f'{self.user.first_name} {self.user.last_name}'

class Address(models.Model)
    customer = models.ForeignKey(Customer, on_delete=models.CASCADE, related_name='addresses')
    ("استان"=max_length=50, verbose_name) province = models.CharField
    ("شهر"=max_length=50, verbose_name) city = models.CharField
    ("آدرس دقیق (خیابان، پلاک، و"=verbose_name) street = models.TextField
    ("احد"
    ("کد پستی"=max_length=10, verbose_name) postal_code = models.CharField
    ("تی

    def __str__(self)
        "return f'{self.province}, {self.city} - {self.postal_code}'
```

11- دستور زیر را در ترمینال تکرار کنید:

```
python manage.py makemigrations
python manage.py migrate
```

خروجی

```
$ python manage.py makemigrations orders
Migrations for 'orders':
  apps\orders\migrations\0001_initial.py
    + Create model Order
    + Create model OrderItem

$ python manage.py migrate
Operations to perform:
  Apply all migrations: accounts, admin, auth, carts, contenttypes, customers, orders, products, sessions
Running migrations:
  Applying orders.0001_initial... OK
```

12- ساخت سریالایزرها: در مسیر `/apps/customers` فایل `serializers.py` را ایجاد کنید:

```
from rest_framework import serializers
from .models import Order, OrderItem
from apps.carts.models import Cart
from .services import OrderService

# ۱. سریالایزر نمایش آیتمهای فاکتور
# 1. Serializer to display invoice items
class OrderItemSerializer(serializers.ModelSerializer):
    # برای نمایش نام محصول به جای فقط آیدی آن
    # To display the product name instead of just its ID
    product_name = serializers.CharField(source='product.name', read_only=True)

    class Meta:
        model = OrderItem
        fields = ['id', 'product_name', 'quantity', 'unit_price']

# ۲. سریالایزر نمایش کل فاکتور
# 2. Serializer to display the entire invoice
class OrderSerializer(serializers.ModelSerializer):
    items = OrderItemSerializer(many=True, read_only=True)

    class Meta:
        model = Order
        fields = ['id', 'status', 'created_at', 'items']

# ۳. سریالایزر عملیاتی: فقط برای دریافت آیدی سبد خرید و ساخت فاکتور
# 3. Operational Serializer: Only for getting the shopping cart ID and creating the invoice
class CreateOrderSerializer(serializers.Serializer):
    cart_id = serializers.UUIDField
```

```

: def validate_cart_id(self, cart_id)
# بررسی اینکه آیا این سبد خرید اصلاً وجود دارد؟
?Check if this shopping cart even exists #
: ( ) if not Cart.objects.filter(id=cart_id).exists
raise serializers.ValidationError
("سبد خرید نامعتبر است یا قبلاً پرداخت شده است.")
return cart_id

: def save(self, **kwargs)
cart_id = self.validated_data['cart_id']

# استخراج مشتری از ریکوئست (کاربر باید لاگین باشد)
# ما request را از طریق context از سمت View به اینجا پاس میده
یم
Extract the customer from the request (user must be logged #
in)
We pass the request here via context from the View side #
customer = self.context['request'].user.customer

# فراخوانی لایه سرویس که در مرحله قبل ساختیم!
!Call the service layer we created in the previous step #
order = OrderService.create_order_from_cart(cart_id=cart_id,
customer=customer)

return order

```

13- ساخت View ها: در مسیر `/apps/customers` فایل `views.py` را باز کنید:
ما به دو API نیاز داریم: یکی برای خواندن و ویرایش پروفایل شخصی، و دیگری برای مدیریت آدرس ها.

```

from rest_framework.viewsets import ModelViewSet
from rest_framework.generics import RetrieveUpdateAPIView
from rest_framework.permissions import IsAuthenticated
from .models import Customer, Address
from .serializers import CustomerProfileSerializer, AddressSerializer

: class CustomerProfileView(RetrieveUpdateAPIView)
"""
این ویو برای مشاهده و ویرایش پروفایل کاربری خود شخص است.
.This view is for viewing and editing a person's own user profile
"""

serializer_class = CustomerProfileSerializer
permission_classes = [IsAuthenticated]

: def get_object(self)

```

```

# این متد جادویی باعث میشود نیازی به ارسال ID در URL نباشد.
# کاربر بر اساس توکنی که میفرستد، فقط پروفایل خودش را دریافت
می‌کند.
This magic method eliminates the need to send the ID in the #
.URL
The user only gets their own profile based on the token the #
.y send
customer, created = Customer.objects.get_or_create(user=self.
request.user)
return customer

:class AddressViewSet(ModelViewSet)
"""
مدیریت آدرسهای پستی کاربر
Manage user mailing addresses
"""
serializer_class = AddressSerializer
permission_classes = [IsAuthenticated]

:def get_queryset(self)
# هر کاربر فقط آدرسهای خودش را می‌بیند
Each user only sees their own addresses #
return Address.objects.filter(customer__user=self.request.use
r)

:def perform_create(self, serializer)
# در زمان ساخت آدرس جدید، فیلد customer به صورت خودکار با کار
بر فعلی پر میشود
When creating a new address, the customer field is automati #
cally filled with the current user
customer, created = Customer.objects.get_or_create(user=self.
request.user)
serializer.save(customer=customer)

```

متد `get_or_create` در ORM جنگو، یک ویژگی خاص دارد: این متد به جای یک آبجکت ساده، همیشه یک **تاپل (Tuple)** دوتایی برمی‌گرداند.

ساختار خروجی این متد به این شکل است: `(object, boolean)`

- **متغیر اول** (`customer`): خود آبجکت دیتابیس است (چه آن را پیدا کرده باشد، چه همان لحظه ساخته باشد).
 - **متغیر دوم** (`created`): یک مقدار منطقی (True یا False) است.
- اگر کاربر قبلاً پروفایل `Customer` داشته و جنگو فقط آن را از دیتابیس خوانده (**Get**) باشد، مقدار `created` برابر با `False` می‌شود.
 - اگر کاربر پروفایل نداشته و جنگو همان لحظه یک رکورد جدید برایش ساخته (**Create**) باشد، مقدار `created` برابر با `True` می‌شود.

چرا آن را نوشتیم ولی در کدهای بعدی از آن استفاده نکردیم؟

در قطعه کدی که من نوشتم، ما عملاً کاری با متغیر `created` نداریم (چون فقط می‌خواستیم پروفایل را به دست بیاوریم). اما **مجبوریم** آن را بنویسیم تا عمل Unpacking (استخراج مقادیر از تاپل) در پایتون به درستی انجام شود. اگر فقط می‌نوشتیم:

```
customer = Customer.objects.get_or_create(user=self.request.user)
```

در این حالت، متغیر `customer` تبدیل به یک تاپل می‌شد (مثلاً: `(Customer Object, False)`) و وقتی در خط بعدی می‌خواستیم آن را به سرالایزر پاس بدهیم، سیستم کُرش می‌کرد و ارور می‌داد چون سرالایزر یک آبجکت می‌خواهد، نه یک تاپل!

دوره استاندارد برای هندل کردن این موضوع در پایتون وجود دارد:

1- روشی که استفاده کردیم (خوانا و واضح برای برنامه‌نویسان دیگر):

```
customer, created = Customer.objects.get_or_create(...)
```

2- استفاده از متغیر پنهان `_` (روش مرسوم در پایتون برای متغیرهایی که نیازی به آن‌ها نداریم):

```
customer, _ = Customer.objects.get_or_create(...)
```

هر دو روش از نظر منطق برنامه‌نویسی کاملاً یکسان عمل می‌کنند.

چه زمانی متغیر `created` واقعاً به درد می‌خورد؟

در سناریوهای تجاری لایه سرویس، این متغیر به شدت کاربردی است. فرض کنید می‌خواهیم وقتی پروفایل مشتری برای اولین بار ساخته می‌شود، یک پیامک یا ایمیل خوش‌آمدگویی برایش ارسال شود. با استفاده از این متغیر به راحتی می‌توانیم این کار را مدیریت کنیم:

```
customer, created = Customer.objects.get_or_create(user=self.request.user)
```

```
if created:
```

```
    # این کد فقط زمانی اجرا می‌شود که رکورد جدیدی در دیتابیس ثبت شده باشد
    # مثلاً: فراخوانی سرویس پیامک
    SmsService.send_welcome_message(customer.user.first_name)
```

پس در کدهای مربوط به این مرحله از توسعه ما، حضور `created` صرفاً برای تفکیک درست خروجی متد جنگو بود تا بتوانیم بدون خطا متغیر `customer` را استخراج کنیم.

14- تنظیم مسیرها: در مسیر `/apps/customers` فایل `urls.py` را ایجاد کنید:

```
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import CustomerProfileView, AddressViewSet

()router = DefaultRouter
router.register('addresses', AddressViewSet, basename='addresses')

] = urlpatterns
path('profile/', CustomerProfileView.as_view(), name='customer-profile')
, path('', include(router.urls))
[
```



تا اینجا در معماری سیستم چه اتفاقی افتاده است؟

ما با موفقیت «ستون فقرات تراکنش‌های مالی» سیستم را ساختیم. در ذهن خود این جریان داده را تجسم کنید:

1. **موجودیت موقت (Cart):** کاربر وارد سایت می‌شود و شروع به انتخاب کالا می‌کند. این اطلاعات در سبد خرید ذخیره می‌شود. سبد خرید بی‌ثبات است؛ کالاها کم و زیاد می‌شوند و قیمت‌ها با نوسان دیتابیس تغییر می‌کنند.
2. **موجودیت دائمی و فریز شده (Order):** به محض اینکه کاربر تصمیم به خرید می‌گیرد، لایه سرویس ما (همان `OrderService`) وارد عمل می‌شود. این لایه مثل یک دروازه‌بان، سبد خرید را از بین می‌برد، موجودی انبار را کسر می‌کند و یک «عکس ثابت و غیرقابل تغییر» از قیمت و کالاها به نام فاکتور (Order) ثبت می‌کند.
3. **هویت مشتری (Customer & Address):** همزمان، ما فضایی ساختیم تا این فاکتورهای بی‌صاحب، به یک انسان واقعی با آدرس پستی دقیق متصل شوند.



راهنمای تست بصری و عملیاتی (قدم به قدم در Swagger)

برای اینکه این مفاهیم انتزاعی را در عمل ببینید، پروژه را `runserver` کنید و آدرس

`http://127.0.0.1:8000/api/schema/swagger-ui/` را در مرورگر باز کنید.

مسیر زیر را دقیقاً به همین ترتیب طی کنید:



قدم اول: احراز هویت (ورود به سیستم)

1. در Swagger به بخش `token` (احتمالاً `/POST /api/token`) بروید.
2. یوزرنیم و پسورد کاربری که قبلاً ساخته‌اید را وارد کنید و `Execute` را بزنید.
3. مقدار `access` توکن را کپی کنید، روی دکمه قفل سبز رنگ (Authorize) در بالای صفحه کلیک کرده و توکن را آنجا قرار دهید. اکنون شما به عنوان یک کاربر لاگین شده شناخته می‌شوید.



قدم دوم: هویت‌بخشی و ثبت آدرس (دستاوردهای جدید)

1. **ساخت/مشاهده پروفایل:** به مسیر `/GET /api/customers/profile` بروید و `Execute` کنید. خواهید دید که سیستم به صورت خودکار (به لطف همان متد `get_or_create`) پروفایل شما را می‌سازد و اطلاعات اولیه را برمی‌گرداند.
2. **ثبت آدرس پستی:** به مسیر `/POST /api/customers/addresses` بروید. در بخش بدنه (Body)، اطلاعات یک آدرس تستی (استان، شهر، خیابان، کد پستی) را وارد و `Execute` کنید. باید کد `Created 201` بگیرید.



قدم سوم: چرخه خرید (آماده‌سازی سبد)

1. یک سبد خرید در `/POST /api/carts` بسازید (آیدی `cart_id` را کپی کنید).
2. یک یا چند محصول تستی را از طریق `/POST /api/cart-items` به این سبد خرید اضافه کنید.



قدم چهارم: لحظه جادویی لایه سرویس (تبدیل سبد به فاکتور)

1. حالا به سراغ مسیر `/POST /api/orders` بروید.
2. در بدنه درخواست، فقط `cart_id` که در مرحله قبل کپی کرده بودید را وارد کنید و `Execute` را بزنید.
3. **نتیجه‌ای که باید ببینید:** یک فاکتور کامل (Order) با وضعیت `P` (در انتظار پرداخت) به شما نمایش داده می‌شود. قیمت‌ها در فیلد `unit_price` ثبت شده‌اند.
4. **تست عمیق‌تر:** اگر الان دوباره سعی کنید سبد خرید قبلی را با متد `GET` فراخوانی کنید، ارور ۴۰۴ می‌گیرید، چون لایه سرویس ما آن سبد را پس از تبدیل به فاکتور، با موفقیت معدوم کرده است! همچنین اگر پنل ادمین محصولات را چک کنید، می‌بینید که موجودی انبار کسر شده است.



مرحله بعدی توسعه چیست؟

ما الان یک فاکتور در انتظار پرداخت داریم و موجودی انبار را هم برای این مشتری رزرو (کسر) کرده‌ایم. در دنیای واقعی کسب‌وکار، قدم منطقی بعدی دریافت پول است!

ایستگاه بعدی ما باید فاز پرداخت (**Payment Domain**) باشد. در این مرحله باید:

- جدولی برای ثبت تراکنش‌های مالی (Payments) بسازیم که به سفارش (Order) متصل باشد.
- معماری اتصال به درگاه پرداخت (مثل زرین‌پال یا یک درگاه شبیه‌ساز تستی) را پیاده‌سازی کنیم.
- پس از پرداخت موفق، وضعیت فاکتور را از **PENDING** به **COMPLETED** تغییر دهیم.



پایان Part-7